Hybrid Cloud Threat Detection & Malware Analysis Platform

Technical Project Report

Author: Mola Etchie
Date: July 20, 2026
Version: 1.0

Executive Summary

This report documents the design, implementation and evaluation of a multi-layer cybersecurity home lab titled Hybrid Cloud Threat Detection & Malware Analysis Platform. The project was built over 24 days on a 2016 MacBook Pro running Ubuntu 26.04 LTS inside a VirtualBox virtual machine, with cloud integration extending into Microsoft Azure. The platform mirrors real-world enterprise Security Operations Centre environments and demonstrates that production-grade security tooling can be deployed, integrated and validated within the constraints of consumer hardware.

The platform was developed across four layers. The first layer established a centralised SIEM foundation using the ELK Stack. The second layer added network intrusion detection through Suricata and Snort. The third layer introduced deep network analysis and malware detection via Zeek and YARA. The fourth and final layer extended the platform into the cloud by integrating Microsoft Azure Activity Logs. By the end of the project, a unified Kibana dashboard was displaying over 23,000 security events drawn from local network traffic, intrusion detection alerts, network behavioural analysis and cloud administrative activity simultaneously.

Background and Motivation

The cybersecurity landscape is increasingly defined by the convergence of artificial intelligence, cloud computing and advanced persistent threats. Organisations require security professionals who can not only operate enterprise tools but architect integrated detection pipelines that span on-premises and cloud environments. This project was undertaken to build practical, hands-on competency in these areas. Rather than completing a guided course or following a tutorial, the goal was to build a real, functioning security platform from scratch and document every challenge encountered along the way.

The project was also motivated by the desire to create a portfolio that demonstrates the kind of troubleshooting, integration and persistence required in real SOC and security engineering environments. Every configuration error, failed service restart and debugging session documented in this report is part of that story.

Project Objectives

The project was designed to achieve the following objectives. First, to deploy and configure a full ELK Stack SIEM environment on a virtualised Ubuntu instance. Second, to integrate network intrusion detection systems capable of identifying and alerting on malicious traffic in real time. Third, to implement malware detection capabilities using YARA signature rules and Zeek deep packet inspection. Fourth, to extend the platform into Microsoft Azure by ingesting cloud Activity Logs into the same centralised monitoring pipeline. Fifth, to simulate real-world attacks and validate detection across all layers. Sixth, to document every challenge encountered and every solution applied as evidence of engineering problem-solving capability.

Operating Environment

The entire platform was built on a 2016 MacBook Pro with 8GB of RAM. Ubuntu 26.04 LTS ran inside a VirtualBox 7.0 virtual machine configured with 4GB of RAM and 2 virtual CPU cores. SSH tunnelling was configured between the Mac host and the Ubuntu VM to enable clipboard sharing and remote terminal access. The cloud layer integrated with Microsoft Azure using the Filebeat Azure module and Azure Event Hub for activity log streaming.

Technology Stack

Technology	Version	Purpose
Ubuntu Server	26.04 LTS	Operating system hosting the security platform
Elasticsearch	8.19.16	Stores and indexes security events
Logstash	8.19.16	Parses and processes collected logs
Filebeat	8.19.16	Collect and forwards logs
Suricata	8.x	Network intrusion detection
Snort	3.x	Signature based intrusion detection
Zeek	8.0.5	Network traffic analysis

YARA	4.5.5	Malware identification using custom rules
Microsoft Azure	N/A	Cloud activity log monitoring

System Architecture

The platform follows a layered architecture where each layer builds on the previous one, adding new detection capabilities while feeding data into a centralised Elasticsearch database. All components communicate through Filebeat log shipping pipelines, with Kibana serving as the unified visualisation layer.

Network traffic enters through the enp0s3 interface of the Ubuntu VM. Suricata and Snort monitor this interface in real time and write alerts to eve.json and alert log files respectively. Zeek captures deep network behavioural data including connection logs, HTTP logs, DNS logs and file transfer logs. YARA runs on a five-minute cron schedule scanning the filesystem for malware signatures and posting results directly to Elasticsearch via the REST API. Filebeat collects logs from all these sources and ships them to Elasticsearch. In parallel, Microsoft Azure streams Activity Logs through an Event Hub, which Filebeat also consumes and forwards to Elasticsearch. Kibana queries Elasticsearch and renders everything across multiple dashboards including the unified Hybrid Threat Detection Platform dashboard.

Implementation — Phase by Phase

Phase 1 — Building the SIEM Foundation

The project began with deploying the ELK Stack completely from scratch on Ubuntu 26.04 LTS. This involved installing Elasticsearch 8.19.16 as the centralised data store, installing Logstash for log processing, installing Kibana for visualisation and configuring Filebeat as the log shipper. Each component required individual configuration including setting up TLS security, configuring authentication credentials, generating Kibana enrollment tokens and verifying that each service could communicate with the others.

A significant early decision was to remove Logstash from the active pipeline and configure Filebeat to send directly to Elasticsearch. This was driven by RAM constraints — running Elasticsearch, Logstash and Kibana simultaneously on a VM with 4GB of RAM caused persistent memory pressure and service instability. Removing Logstash from the live pipeline freed approximately 500MB of RAM and significantly improved stability. Logstash remained installed and configured for documentation purposes but Filebeat was reconfigured with direct Elasticsearch output.

By the end of this phase, centralised log collection was working. Linux system logs, syslog entries and authentication logs were being collected by Filebeat and indexed in Elasticsearch. A custom index pattern was created in Kibana and the first data view was confirmed working.

Phase 2 — Log Collection and Pipeline Configuration

With the SIEM foundation in place, the focus shifted to log collection coverage. Filebeat was configured to collect Linux system logs, authentication logs and network logs. The system module and the custom filestream input were both configured and tested. Logstash pipelines were written to parse raw syslog entries using Grok patterns, normalise field names and route structured data into Elasticsearch indices named by date.

Several pipeline errors were encountered during this phase including incorrect Grok syntax, wrong file paths and permission errors on log files. Each error was resolved by testing the pipeline configuration using the Logstash config test flag before restarting the service.

Phase 3 — Dashboard Development

Five Kibana dashboards were created during the project. The Network Security Monitor dashboard was the first custom dashboard built, containing four visualisations including a bar chart of log events over time, a pie chart of response codes, a data table of top source IPs and a metric panel showing total event count. The Hybrid Threat Detection Platform dashboard was the final unified dashboard built during Day 23 and displayed data from all four layers simultaneously.

Building dashboards required determining the correct data views, writing appropriate KQL filter queries, selecting the correct aggregation types and validating that each visualisation was pulling from the correct index and time window. Each visualisation had to be validated individually before being added to the dashboard.

Phase 4 — Detection Rules and Alerting

Detection logic was implemented inside Kibana using Elasticsearch query rules. A failed SSH login detection rule was configured to trigger when three or more failed authentication events were detected within a five-minute window. The Kibana encryption key was added to the Kibana configuration file to enable the alerting feature, which required a service restart and re-enrollment. Threshold settings were refined to reduce noise while maintaining meaningful alert sensitivity.

Phase 5 — Suricata IDS Integration

Suricata 8.0.5 was installed from the OISF stable PPA and configured to monitor the enp0s3 network interface. The Suricata configuration file was updated with the correct interface name

and rule paths. The Filebeat Suricata module was enabled and configured to read from the eve.json log file. Filebeat setup was run to load pre-built Suricata dashboards into Kibana.

Suricata integration involved an extended debugging period. Flow events were appearing in Elasticsearch but alert events were not. The issue was traced to rule loading — Suricata was running but the rule files were not being correctly loaded on startup. After correcting the rule file paths in the configuration and restarting the service, alerts began appearing. Over 15,000 Suricata security events were eventually collected including flow events, DNS traffic, HTTP events and intrusion detection alerts.

Phase 6 — Zeek Deep Network Analysis

Zeek 8.0.5 was installed from the OpenSUSE security repository. One of the key learnings from this phase was that Zeek is not managed through systemd. Attempting to run sudo systemctl start zeek returned a unit not found error. Zeek is managed through its own control tool, zeekctl, using the deploy command. Once this was understood, Zeek was deployed successfully on enp0s3 and began generating connection logs, HTTP logs, DNS logs and file transfer logs immediately.

The Filebeat Zeek module was enabled and configured with paths pointing to the Zeek current logs directory. A YAML indentation error in the zeek.yml module file caused Filebeat to crash several times before the correct syntax was established. Once resolved, Zeek data flowed into Elasticsearch and the pre-built Zeek Filebeat dashboards were loaded, including the geo-destination world map showing real-time network connection origins.

Phase 7 — YARA Malware Detection

YARA 4.5.5 was installed and three custom detection rules were written. The first rule, SuspiciousStrings, detects common malware command strings including cmd.exe, powershell and base64_decode. The second rule, PhishingKeywords, detects credential harvesting language including phrases related to passwords, credit cards and account verification. The third rule, MalwarePatterns, detects PE file header bytes and dangerous Windows API calls including CreateRemoteThread, VirtualAlloc and WinExec.

A custom bash script was written to run YARA scans automatically every five minutes using a cron job. The script scans the filesystem, captures any rule matches and posts the results directly to a dedicated yara-alerts index in Elasticsearch via the REST API using curl. All three rules were validated against test files containing the relevant patterns. Twelve YARA alerts were generated and confirmed visible in Kibana Discover.

Phase 8 — Azure Cloud Integration

Azure integration became an entire project within the project. The process involved creating an Azure account, setting up an Event Hub namespace, creating an Event Hub named

insights-operational-logs, configuring a Storage Account for Filebeat state management, enabling Azure Activity Log diagnostic settings to stream to the Event Hub and configuring the Filebeat Azure module with the Event Hub connection string and storage account credentials.

Several problems were encountered including YAML indentation errors in the azure.yml module file that caused Filebeat to crash repeatedly, authentication issues with the storage account name, and initial confusion about whether logs were flowing because the default Kibana time filter of 24 hours excluded older events. Switching the time filter to 30 days confirmed that Azure logs had been flowing correctly all along. Generating fresh Azure activity including resource group creation and deletion populated the dashboard with current-day events. Seven Azure Activity Log documents were confirmed in Elasticsearch including administrative events showing Microsoft.Resources subscription and resource group operations.

Phase 9 — Unified Hybrid Dashboard

The final dashboard brought everything together. The Hybrid Threat Detection Platform dashboard was created with four panels displaying data from all integrated sources simultaneously. The Total Events panel showed a combined count of 23,774 security events. The Suricata Alerts panel displayed a bar chart of alert signatures. The Azure Activity Logs panel showed cloud administrative operations including resource group deletions. The Zeek Network Connections panel showed network transport protocol distribution across UDP, TCP and ICMP traffic. This dashboard represents the capstone of the entire project — a single view across local network security, intrusion detection and cloud activity.

Phase 10 — Attack Simulation and Validation

Four attack simulations were conducted to validate end-to-end detection. SSH brute force was simulated by repeatedly attempting failed logins against the local SSH service, triggering the Kibana alert rule. A port scan was simulated using nmap against localhost, with Suricata detecting and logging the scanning activity. General network activity was generated using ping and curl commands, captured by both Suricata and Zeek. Cloud administrative actions were simulated in Azure by creating and deleting resource groups, which were captured in the Azure Activity Logs and appeared on the unified dashboard.

Major Problems Encountered and Solutions Applied

Problem 1 — Kibana Memory Pressure and Startup Failures

Running Elasticsearch, Logstash, Kibana, Filebeat, Suricata and Zeek simultaneously inside a VM with 4GB of allocated RAM was a persistent challenge. Kibana was the most sensitive service, frequently failing to start or becoming unresponsive under memory pressure. The solution was a combination of stopping non-essential services before starting Kibana, removing

Logstash from the active pipeline to free 500MB of RAM and establishing a consistent startup sequence that started Elasticsearch first, waited for it to fully initialise before starting Kibana, and only opened the SSH tunnel after both services were confirmed running.

Problem 2 — Logstash Pipeline Errors

Multiple pipeline errors were encountered during Logstash configuration including wrong Grok syntax, incorrect file paths, missing permissions on log files and pipelines that would pass the config test but fail on startup. Each was resolved by using the config test and exit flag to validate configuration before restarting and checking journalctl logs for the specific error line.

Problem 3 — Filebeat Module YAML Indentation

YAML is whitespace-sensitive and Filebeat module configuration files caused repeated crashes due to indentation errors. The zeek.yml file in particular had var.paths lines that were not correctly indented under their parent keys, causing Filebeat to exit with an invalid config error. The solution was to delete the problematic module file and rewrite it from scratch with careful attention to indentation.

Problem 4 — Zeek Not Found in Systemd

Attempting to manage Zeek through systemctl returned a unit not found error. The discovery that Zeek uses its own management tool zeekctl rather than systemd was a significant learning point. Once understood, deployment, status checking and restarting all used zeekctl commands instead.

Problem 5 — Suricata Alerts Not Appearing

Flow events were visible in Elasticsearch but alert events were not. After an extended debugging session the issue was traced to rule loading — Suricata was running but not correctly loading its rule files. Correcting the rule paths in the Suricata configuration file and restarting the service resolved the issue and alerts began appearing.

Problem 6 — Azure Time Filter Confusion

After configuring the Filebeat Azure module and waiting for logs to appear, the Kibana dashboard showed no Azure data. The issue was that Azure logs had been collected but fell outside the default 24-hour time filter. Switching to a 30-day time range confirmed the logs existed and were correctly indexed. Generating fresh Azure activity then populated the dashboard with current-day events.

Problem 7 — Azure Filebeat Module Storage Account Error

The Filebeat Azure module crashed repeatedly with an error stating that the storage account value was not set. Investigation revealed that the storage account name in the azure.yml configuration file did not exactly match the actual Azure storage account name. Correcting the name to match the exact Azure resource name resolved the issue.

Problem 8 — Cloud Account Payment Validation

Both AWS and Microsoft Azure rejected Nigerian Mastercard during account creation. This was resolved by obtaining a virtual international dollar card through a Nigerian fintech provider, which was accepted successfully by Azure.

Problem 9 — VirtualBox Clipboard Sharing

Copy-paste between the Mac host and the Ubuntu VM failed due to VirtualBox Guest Additions incompatibility on macOS Monterey. This was resolved by setting up an SSH tunnel from Mac Terminal to the Ubuntu VM, which allowed commands to be pasted directly from Claude into the Mac Terminal and executed inside Ubuntu. This approach also reflected professional server management practice.

Problem 10 — Ubuntu 26.04 GPG Key Incompatibility

The apt-key command was removed in Ubuntu 26.04, preventing the addition of the Elastic repository using the standard method. The solution was to use the gpg dearmor method with the signed-by parameter in the apt sources list, which is the correct approach for modern Ubuntu versions.

Results

By the end of the project the platform was fully operational across all four layers. Elasticsearch contained indices for Linux system logs, Suricata alerts, Zeek network logs, YARA malware alerts and Azure Activity Logs. The unified Hybrid Threat Detection Platform dashboard displayed 23,774 combined security events. Suricata had detected over 15,000 security events. YARA had generated 12 malware detection alerts. Zeek had captured over 4,800 network documents. Seven Azure Activity Log events were confirmed in Elasticsearch. Five Kibana dashboards were created and operational. Four attack simulations were conducted and validated.

Skills Demonstrated

The project provided practical experience across a wide range of cybersecurity and engineering disciplines. In SIEM engineering the project covered ELK Stack deployment, Elasticsearch configuration, Logstash pipeline development, Kibana dashboard building and Filebeat configuration. In Linux administration it covered Ubuntu server management, systemd service management, YAML configuration, JSON log analysis and SSH tunnel configuration. In network

security monitoring it covered intrusion detection system deployment, network traffic analysis and deep packet inspection. In threat detection engineering it covered log analysis, event correlation, detection rule development and security visualisation. In cloud security monitoring it covered Azure Monitor configuration, Azure Activity Log integration and Event Hub setup. In troubleshooting it covered debugging distributed systems, tracing errors across multiple interconnected services and resolving dependency conflicts.

Lessons Learned

Hardware constraints drive architectural decisions. Removing Logstash from the active pipeline was not a workaround — it was an architectural optimisation that improved stability and reflects real-world decisions made in resource-constrained environments.

SSH-based server management is the professional standard. Setting up the SSH tunnel solved the clipboard problem but also meant every command was being executed through a proper remote terminal session rather than a GUI clipboard hack.

Logs always exist somewhere. The Azure time filter confusion taught an important lesson — before concluding that integration has failed, verify the time window. Data that appears missing is often simply outside the current view.

Iterative development with snapshots saves enormous amounts of time. Taking VirtualBox snapshots at the end of each day meant that any catastrophic failure could be recovered from without rebuilding the entire platform.

Persistence is a technical skill. Many of the problems in this project had no obvious solution in the documentation. The debugging process — forming a hypothesis, testing it, observing the result and iterating — is itself a professional capability that this project demonstrates repeatedly.

Future Improvements

Several improvements are planned for future iterations of the platform. Adding machine learning anomaly detection using Elastic ML would allow the platform to identify unusual patterns without requiring explicit rule definitions. Expanding the YARA rule library to cover additional malware families and APT indicators would increase detection coverage. Implementing automated incident response playbooks triggered by Kibana alert thresholds would move the platform toward active response rather than passive detection. Integrating MISP for structured threat intelligence sharing would add professional IOC lifecycle management. Upgrading the VM to 16GB of RAM would allow the full Logstash pipeline to be restored and additional services to run simultaneously without memory pressure.

References

Elastic. (2026). Elasticsearch 8.x Documentation. https://www.elastic.co/guide/en/elasticsearch/reference/current

OISF. (2026). Suricata Documentation. https://docs.suricata.io

Snort.org. (2026). Snort 3 User Manual. https://www.snort.org/documents

Zeek Project. (2026). Zeek Documentation. https://docs.zeek.org

VirusTotal. (2026). YARA Documentation. https://yara.readthedocs.io

Microsoft. (2026). Azure Monitor Activity Log. https://learn.microsoft.com/azure/azure-monitor/essentials/activity-log

MITRE. (2026). ATT&CK Framework. https://attack.mitre.org